

Generative Modelling of Maritime AIS Trajectories

An Iterative Engineering Study across Twelve Model Versions

Andrei Știrbu

École Polytechnique

Research Internship Report

May 2026

Abstract

Given a start position, a destination, and a vessel type, this report addresses the problem of generating a 128-point ship route that is geographically admissible (wholly on water), behaviourally plausible (resembling real commercial traffic), and geometrically sound (smooth, correct total length, zero endpoint error). Twelve model iterations were designed, trained, and evaluated on twelve months of US-region NOAA Automatic Identification System (AIS) data from 2024. The study spans next-step displacement prediction (v1), an encoder-decoder trajectory generator (v2–v5), a parked obstacle-avoidance branch (v6–v8), retrieval-augmented generation (v9), per-timestep retrieval with land-aware training (v10), and two parked pointer-based attempts (v11–v12). The production system—v10 at epoch 52 coupled with a WaterRouter snap-only post-processor evaluated on a land-filtered dataset—achieves **Average Displacement Error (ade) of 6282 m** and **0 % land-crossing** at the 10 km threshold ($n = 500$, seed = 42). The headline architectural finding is that almost every measurable improvement came from data-level or post-processing changes rather than from new model components: per-step accuracy gains across v2–v5 produced no real-world ADE movement, retrieval (v9) was the sole architectural innovation that moved the needle, and the largest single ADE reduction (−21 %) came from cleaning the training dataset rather than from any model change. A thirteenth iteration—a conditional denoising diffusion Transformer (TrajDiff)—is fully specified but not yet implemented.

Contents

1	Introduction and Context	4
1.1	Maritime trajectory modelling	4
1.2	Problem statement	4
1.3	Why this is hard	4
1.4	Contribution overview	5
2	Background	5
2.1	Transformer encoder and encoder-decoder	5
2.2	Autoregressive generation and exposure bias	5
2.3	Denoising diffusion models	5
2.4	Signed-distance fields	6
3	Dataset and Experimental Setup	6
3.1	Data source and filtering	6
3.2	Evaluation protocol	6
3.3	Hardware and software	7
3.4	Baselines	7

4	Technical Contributions	7
4.1	v1 – Next-step displacement prediction	7
4.1.1	Architecture	7
4.1.2	Result	7
4.1.3	Lesson	8
4.2	v2–v5 – Encoder-decoder trajectory generator	8
4.2.1	Reframing as seq2seq	8
4.2.2	Architecture (<code>src/model_gen.py</code>)	8
4.2.3	Linear endpoint correction	9
4.2.4	Scheduled sampling (v3, v5)	9
4.2.5	Architectural ladder results	9
4.3	v6, v7, v8 – Obstacle-conditioned generation (parked)	9
4.3.1	Motivation and architecture	10
4.3.2	v6 — the data leak	10
4.3.3	v7 — the structural failure	10
4.4	v9 – Retrieval-augmented generation	11
4.4.1	Retrieval-top-1 gate	11
4.4.2	Architecture (<code>src/model_gen_retrieval.py</code>)	11
4.4.3	Results	11
4.5	v10 – Per-timestep retrieval and land-aware training	11
4.5.1	Change 1: Per-step retrieval bank	11
4.5.2	Change 2: Windowed causal mask	12
4.5.3	Change 3: Land-aware loss and hard-projection inference	12
4.5.4	Results on dirty data	13
4.6	WaterRouter post-processor	13
4.7	E1 – Upstream data cleaning	14
4.7.1	Diagnosis	14
4.7.2	The filter	14
4.7.3	Impact	14
4.8	v11 – Halton-cone pointer (parked)	15
4.9	v12 – Water-cell K-ring pointer (abandoned)	15
5	Results and Discussion	15
5.1	Summary of all trajectory-generation experiments	15
5.2	Cross-cutting findings	16
5.3	Production system use-case ranking	16
6	Summary and Conclusion	16
6.1	What was achieved	17
6.2	Open challenges	17
6.3	The project’s deepest lesson	18
7	Outlook: Proposed v13 – TrajDiff	18
7.1	Motivation	18
7.2	Architecture overview	19
7.3	Length router (v13a)	19
7.4	Pre-flight checklist	19
A	Reproducibility Notes	20
B	Repository Structure	21

C	Missing or Incomplete Visual Evidence	22
C.1	Multi-seed variance plot	22
C.2	v13 TrajDiff sample diversity plot	22
C.3	v13 length router evaluation	22
C.4	Architecture diagrams for v9, v12, v13	22

1 Introduction and Context

1.1 Maritime trajectory modelling

Shipping carries over 80 % of world trade by volume, making maritime route intelligence a critical infrastructure. The Automatic Identification System (AIS) mandates that commercial vessels above 300 GT broadcast their position, speed, heading, and vessel category at intervals of a few seconds to several minutes. The US coastal coverage of the MarineCadastre dataset [7] logs approximately 170,000 vessels per day in the bounding box $[\text{lon } -125^\circ, -60^\circ] \times [\text{lat } 10^\circ, 55^\circ]$, providing a rich empirical record of actual traffic patterns.

Downstream consumers of route models include: route planners (planning fuel-efficient voyages), ETA estimators (forecasting port-arrival times), anomaly detectors (flagging routes that deviate from historical patterns), and digital-twin platforms such as (author?) [2], which require a synthetic route generator to stress-test event detectors. A survey of trajectory-prediction methods, from Kalman filters to recurrent networks to Transformer encoders, is given in (author?) [6].

1.2 Problem statement

Formally, given

$$(\text{start} = (\text{lon}_0, \text{lat}_0), \quad \text{end} = (\text{lon}_T, \text{lat}_T), \quad v \in \{0, \dots, 27\})$$

the system must emit a sequence of 128 geographic waypoints

$$\mathbf{x} = (x_1, \dots, x_{128}), \quad x_i = (\text{lon}_i, \text{lat}_i) \in \mathbb{R}^2,$$

with $x_1 \approx \text{start}$, $x_{128} \approx \text{end}$, and $\mathbf{x}$ resembling a plausible commercial shipping route. Plausibility has three levels in decreasing strictness: (1) *geographic admissibility* — the route lies on water; (2) *behavioural fidelity* — the route follows patterns typical for vessel type v ; (3) *geometric quality* — the route is smooth and has correct total length.

1.3 Why this is hard

Three properties of the problem made this project longer than expected, and understanding them is essential to follow the design choices in Section 4.

Irregular constraint surface. Coastlines are GSHHS shoreline polygons [11] rasterised onto a grid; near-coast cells flip from “land” to “water” over ~ 5 km, and many narrow inlets are technically water but commercially unreachable. Any model that attempts to enforce water-validity via a smooth loss is fighting against a discontinuous constraint.

Multimodal route distribution. A ship from Miami to Galveston can plausibly hug the Gulf coast, cut directly across the Gulf of Mexico, or avoid an offshore platform. AIS data records exactly one of these choices per voyage. A point-prediction model trained on this data will reproduce one mode—or worse, an average between two modes that is itself implausible—and the standard ADE metric measures distance to *one* ground-truth, not coverage of the mode set.

One-hundred-fold length variance. Straight-line distances in the dataset range from 20 km to 2000 km. Short routes are nearly straight and reward the great-circle baseline; long routes look like multi-leg voyages and reward retrieval-style memorisation. No single inductive bias serves both.

Noisy ground truth. The original dataset, built without coastal filtering, had ground-truth tracks that crossed land 36 % of the time at a 10 km SDF threshold and 87 % at a 0 km threshold—not a model bug, but a data issue arising from rasterisation noise and genuine coastal-river traffic. Cleaning the dataset (Section 4.7) turned out to be the largest single-step improvement in the entire project.

1.4 Contribution overview

This report documents twelve model iterations and proposes a thirteenth. The student’s original contributions are:

- The full data pipeline from NOAA AIS zips to 128-point route NPZ datasets (including the E1 land-filtering step, Section 4.7).
- Twelve distinct model architectures/training procedures (v1 through v12), each with training, evaluation, and post-mortem analysis.
- The WaterRouter post-processor (Section 4.6): A*- and BFS-based snapping on a fine-resolution water-cell graph.
- The land signed-distance field infrastructure (Section 4.5.3) used for differentiable training loss and hard projection.
- The v13 TrajDiff engineering specification (Section 7).

Background material (Transformer architecture, DDPM, GSHHS, A* search) is cited throughout and clearly distinguished from original work.

2 Background

2.1 Transformer encoder and encoder-decoder

The Transformer [10] replaced recurrent networks as the standard sequence-modelling primitive. A *Transformer encoder* is a stack of identical layers; each layer applies multi-head self-attention followed by a position-wise feed-forward network (FFN). Self-attention computes, for each token i , a weighted sum of all tokens’ values:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where Q, K, V are query, key, and value projections. A causal (triangular) mask restricts each token to attend only to past tokens, enabling autoregressive generation.

An *encoder-decoder* Transformer pairs an encoder that produces a context *memory* with a decoder that generates outputs token-by-token through cross-attention to that memory. This architecture underlies v2–v10 of this project.

2.2 Autoregressive generation and exposure bias

In autoregressive generation, the decoder produces position $t + 1$ from all previously generated positions. During training with *teacher forcing*, the decoder receives ground-truth positions as input. At inference, it receives its own predictions, creating a distribution mismatch (exposure bias) that compounds errors over long rollouts. *Scheduled sampling* [1] mitigates this by probabilistically substituting self-predictions for ground-truth inputs during training.

2.3 Denoising diffusion models

Denoising Diffusion Probabilistic Models (DDPM) [4] define a *forward process* that gradually adds Gaussian noise to a data sample x_0 :

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I),$$

where $\bar{\alpha}_t$ follows a cosine schedule over $T = 1000$ steps. A neural network learns to predict ε (the “noise”) given x_t and t . Denoising Diffusion Implicit Models (DDIM) [9] accelerate sampling

to 20–50 steps by reusing predicted-noise estimates across timesteps. Classifier-free guidance (CFG) [5] mixes conditional and unconditional noise predictions at inference:

$$\hat{\varepsilon} = \varepsilon_{\text{uncond}} + w(\varepsilon_{\text{cond}} - \varepsilon_{\text{uncond}}), \quad w > 1,$$

steering the sample toward a conditioning signal. Diffusion Transformers (DiT) [8] replace U-Net backbones with Transformer blocks modulated by adaptive layer normalisation (adaLN-Zero).

2.4 Signed-distance fields

A signed-distance field (SDF) assigns to each point $p \in \mathbb{R}^2$ a scalar $\text{sdf}(p)$ whose absolute value is the shortest distance to a reference boundary ∂S , and whose sign indicates which side of the boundary p is on. In this project, **positive** values indicate land (inside the GSHHS polygon S) and **negative** values indicate water. The SDF is precomputed once from the GSHHS shoreline [11] at 0.05° resolution (Section 4.5.3).

3 Dataset and Experimental Setup

3.1 Data source and filtering

The dataset is twelve months of US-region NOAA AIS broadcasts from 2024, covering vessel types 60–89 (passenger, cargo, tanker, and similar commercial classes) within the bounding box $\text{lon} \in [-125^\circ, -60^\circ]$, $\text{lat} \in [10^\circ, 55^\circ]$. Raw pings are filtered by `prepare_data.py`: minimum track length 50 pings, maximum inter-ping jump 2 km, maximum gap 60 min, turn-segmentation at headings exceeding 150° , minimum average speed 1 km/h, minimum total distance 5 km. Approximately 97 % of raw pings are removed (80 % by vessel-type filter alone).

Tracks are resampled to exactly 128 points by uniform arc-length interpolation using `prepare_trajgen.py`. This produces a dataset of 207,000 routes stored as `trajgen_128.npz`; each entry is a (128×2) array of normalised (lon, lat) values paired with metadata (start, end, vessel type). A cleaned subset `trajgen_128_clean.npz` is described in Section 4.7.

3.2 Evaluation protocol

All trajectory-generation experiments are evaluated on a fixed set of $n = 500$ validation routes sampled with `seed=42`. The train/validation split is by *root MMSI*: all AIS segments of the same physical vessel (identified by stripping batch prefix and segment suffix from the synthetic vessel ID) land entirely in train or entirely in validation, preventing data leakage.

Two primary metrics are reported throughout:

- **ade (Average Displacement Error)**: mean Euclidean distance in metres between predicted and ground-truth waypoints, averaged over all 128 steps and all 500 validation routes.
- **fde (Final Displacement Error)**: Euclidean distance in metres between predicted and ground-truth *final* waypoints. Linear endpoint correction (Section 4.2.3) drives FDE to near zero for all AR models.

Length-bucketed ADE further breaks down results into short (< 50 km), medium (50–500 km), and long (> 500 km) route categories.

Land-crossing metrics are: `land%@d` (fraction of predicted waypoints with $\text{sdf} > d$ km) and `cross%@d` (fraction of trajectories with at least one waypoint exceeding the threshold).

3.3 Hardware and software

Training runs were executed on heterogeneous hardware (CUDA GPU, Apple Silicon MPS, and CPU), handled by device auto-detection logic consistent across all model versions. All models are implemented in PyTorch. Inference latency for v10 + WaterRouter snap-only is approximately 50 ms per query on a single CPU core.

3.4 Baselines

Great-circle (GC): 128 uniformly-spaced points along the geodesic arc between start and end. ADE = 8226 m on `trajgen_128.npz`; serves as the lower-effort geometric baseline throughout.

Retrieval-top-1: returns the single most similar training route verbatim, selected by 5-D KNN over (start, end, vessel-type). ADE = 6470 m on `trajgen_128.npz` (seed=42 evaluation harness); a zero-training baseline showing the information value of historical traffic.

4 Technical Contributions

4.1 v1 – Next-step displacement prediction

4.1.1 Architecture

The first model treated trajectory modelling as a next-step regression problem: given the past $L = 120$ ping triples $(lon, lat, \Delta t)$, predict the displacement $(\Delta lon, \Delta lat)$ to the next ping. The implementation is a Transformer encoder with a causal (triangular) mask (`src/model.py`, class `AISTransformer`). Each timestep is encoded as a 5-dimensional vector:

$$x_i = [lon_i^{\text{norm}}, lat_i^{\text{norm}}, \log(\Delta t_i)^{\text{norm}}, \Delta lon_i^{\text{norm}}, \Delta lat_i^{\text{norm}}]$$

plus a learned 8-dimensional vessel-type embedding added at every step. Displacement is chosen as the regression target rather than absolute position because it has small, zero-centred variance independent of geographic location, keeping the per-coordinate target distribution stable across the dataset. Three independent scalars (position, log- Δt , displacement) are fitted on the training partition only.

The *bidirectional variant* (mean pooling of encoder outputs) was also implemented and immediately caught: it achieved much lower training MSE by attending to future positions, producing perfect training curves but failing at inference. **Never use bidirectional attention for causal prediction**; this finding is recorded as an architectural warning in the project documentation.

4.1.2 Result

Table 1: v1 next-step model vs. constant-velocity (CV) baseline. ADE in metres; lower is better.

Run	CV ADE	Model ADE
12-month full dataset	50.7	50.8
Open-sea filtered	47.0	47.5
Open-sea + lighthouse	47.0	48.6

The model matches but never beats the constant-velocity (CV) baseline (Table 1). A multi-step rollout evaluation confirmed divergence faster than CV at horizons 5, 10, and 20. Adding lighthouse-distance features made results strictly worse.

4.1.3 Lesson

Open-sea AIS tracks are nearly linear at 3-minute pings: the CV predictor is near-optimal on post-turn-segmented data. More broadly, **per-step accuracy in normalised MSE is not predictive of trajectory-level error**. This finding recurs repeatedly through v2–v12 and is the single most important cross-cutting lesson of the project.

4.2 v2–v5 – Encoder-decoder trajectory generator

4.2.1 Reframing as seq2seq

The pivot from v1 to v2 reframed the task as a *sequence-to-sequence* generation problem: given (start, end, vessel.type), generate a full 128-point route. The dataset was rebuilt by `prepare_trajgen.py`, which resamples each track to exactly 128 arc-length-uniform waypoints and discards time-delta information (vessel speed is recovered implicitly through the vessel-type embedding).

4.2.2 Architecture (`src/model_gen.py`)

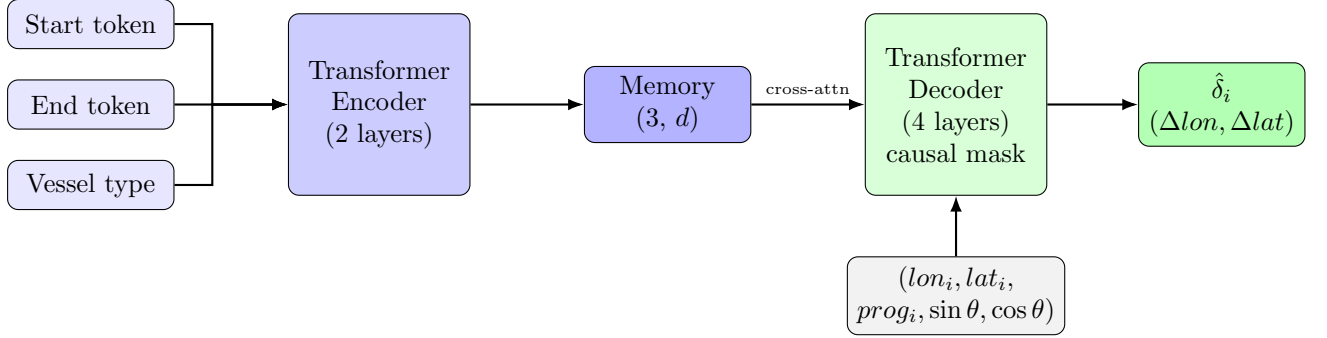


Figure 1: v4/v5 encoder-decoder architecture. The encoder produces 3 context tokens from start, end, and vessel-type conditioning. The decoder generates the 128-point route autoregressively via causal self-attention and cross-attention to the encoder memory. Input features include bearing-to-destination ($\sin \theta, \cos \theta$) added in v4.

Encoder. Three conditioning tokens (start, end, vessel-type), each projected through a linear layer to d_{model} , plus learned token-type embeddings to distinguish their roles. Output: memory $\in \mathbb{R}^{3 \times B \times d_{\text{model}}}$.

Decoder. Consumes a per-step input and generates 128 waypoints autoregressively. Each step’s input (3-dim for MVP/v2/v3; 5-dim for v4/v5 with bearing) is projected to d_{model} , summed with sinusoidal positional encoding, and passed through four Transformer decoder layers (causal self-attention + cross-attention to encoder memory). A linear head emits predicted displacement $(\Delta lon, \Delta lat)$; the next position is recovered as:

$$\text{pos}_{i+1} = \text{pos}_i + \delta_{\text{scaler}}^{-1}(\hat{\delta}_i).$$

Loss.

$$\mathcal{L} = \mathcal{L}_{\delta} + \lambda_{\text{end}} \mathcal{L}_{\text{endpoint}} + \lambda_{\text{smooth}} \mathcal{L}_{\text{smooth}}, \quad (1)$$

where $\mathcal{L}_{\delta}$ is Huber loss on predicted vs. ground-truth deltas, $\mathcal{L}_{\text{endpoint}} = \|\text{pos}_{\text{pred}}^T - \text{pos}_{\text{gt}}^T\|^2$, and $\mathcal{L}_{\text{smooth}} = \frac{1}{T-2} \sum_i \|\text{accel}_i\|^2$ penalises second-order acceleration. MVP configuration: $\lambda_{\text{end}} = 10$, $\lambda_{\text{smooth}} = 1.0$.

4.2.3 Linear endpoint correction

After autoregressive rollout, the residual endpoint error $r = \text{pos}_{\text{gt}}^T - \text{pos}_{\text{pred}}^T$ is distributed linearly across all 128 waypoints:

$$\text{pos}_i^{\text{corrected}} = \text{pos}_i^{\text{raw}} + r \cdot \frac{i}{T-1}, \quad i = 1, \dots, 128.$$

This drives FDE to near zero without retraining and is retained in every subsequent model.

Algorithm 1 Linear Endpoint Correction

Require: raw trajectory $\mathbf{x}^{\text{raw}}$, ground-truth endpoint pos_{gt}^T

```

1:  $r \leftarrow \text{pos}_{\text{gt}}^T - x_{128}^{\text{raw}}$ 
2: for  $i = 1$  to 128 do
3:    $x_i^{\text{corrected}} \leftarrow x_i^{\text{raw}} + r \cdot (i - 1) / (128 - 1)$ 
4: end for
5:
6: return  $\mathbf{x}^{\text{corrected}}$ 
```

4.2.4 Scheduled sampling (v3, v5)

To mitigate exposure bias, v3 and v5 use *scheduled sampling* [1]. The naive step-by-step replacement caused GPU OOM at $T = 128$ due to sequential forward passes. The implementation uses a *two-pass parallel* scheme: Pass 1 runs the standard parallel teacher-forced decoder under `torch.no_grad()` to obtain one-step-ahead predictions; a mixed input trajectory is constructed by selecting ground-truth or self-prediction per step with probability p_{ss} ; Pass 2 re-runs the decoder with gradients on the mixed input. Cost: $\approx 2\times$ standard teacher forcing vs. $\sim 128\times$ for the naive loop.

4.2.5 Architectural ladder results

Table 2: v2–v5 architectural ladder. val_delta is per-step loss in normalised delta space; ADE is evaluated by autoregressive rollout over 500 validation routes (seed=42). The architectural ladder produced no real-world ADE improvement.

Config	Changes vs. MVP	d_{model}	Enc. layers	val_delta	ADE (m)
MVP	Baseline	128	0	0.086	6807
v2	$\lambda_{\text{end}} = 50$	128	0	0.105	—
v3	v2 + SS	128	0	0.105	—
v4	$d=256$, bearing feat., 2 enc. layers	256	2	0.072	—
v5	v4 + SS	256	2	0.074	6753

Figure 2 and Table 2 document the central negative finding: a 14 % improvement in val_delta ($0.086 \rightarrow 0.074$) corresponds to a $\sim 1\%$ improvement in ADE ($6807 \rightarrow 6753$ m) that vanishes inside evaluation noise. The entire architectural ladder produced no measurable real-world improvement.

4.3 v6, v7, v8 – Obstacle-conditioned generation (parked)

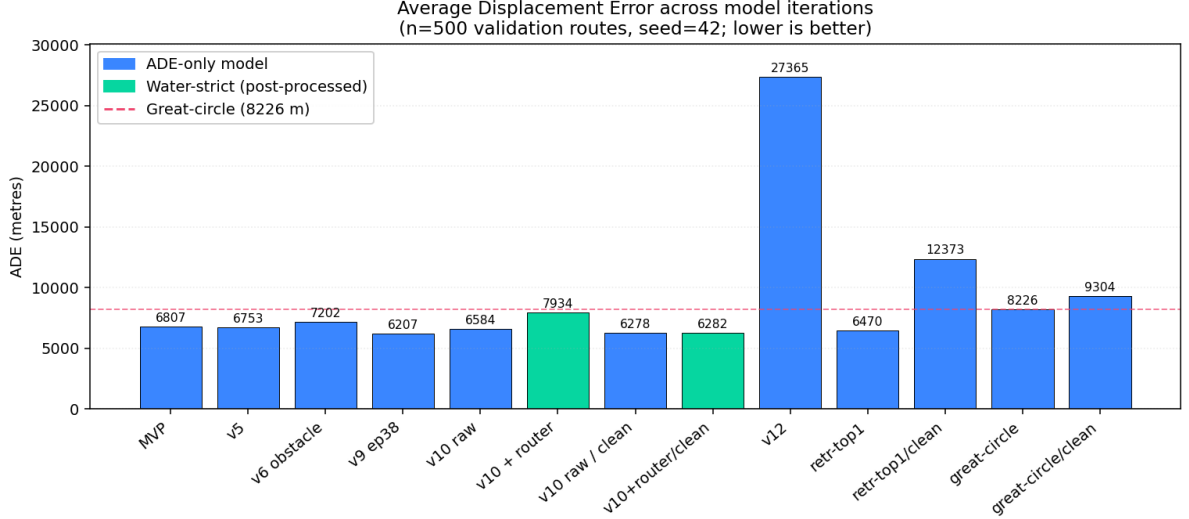


Figure 2: ADE across model iterations. Blue bars: ADE-only models on `trajgen_128.npz` (dirty data). Green bars: water-strict variants. Dashed red line: great-circle baseline. Note the logarithmic scale on the right portion of the x-axis for v12.

4.3.1 Motivation and architecture

v6/v7 extended the v4/v5 encoder with $K \leq 3$ obstacle tokens, each encoding (centre_lon, centre_lat, radius_deg) projected to d_{model} . The training loss added a fourth term:

$$\mathcal{L}_{\text{obs}} = \frac{1}{T} \sum_i \max(0, r + m - \|x_i - c\|)^2, \quad (2)$$

a one-sided quadratic penalty firing when predicted waypoint x_i penetrates obstacle circle (centre c , radius r , margin m). The intent was obstacle-avoidance: given exclusion zones, the decoder should route around them.

4.3.2 v6 — the data leak

`ObstacleTrajectoryGenDataset.__getitem__` seeded its RNG with `self.seed + idx`, assigning the *same* obstacle placement to every sample at every epoch. The model memorised (trajectory, obstacle) pairs; the ground-truth route never crossed those obstacles; $\mathcal{L}_{\text{obs}} \equiv 0$ throughout training. The training log showed `val_obstacle` $\approx 3 \times 10^{-5}$ —not “avoidance working”, but “the penalty had nothing to penalise”. Evaluation returned an avoidance rate of 0.0%.

4.3.3 v7 — the structural failure

Fixing the data leak (`deterministic=False` in the dataset) revealed a deeper problem: the obstacle augmenter placed obstacles *laterally offset* from the ground-truth route, not on it. The GT route therefore never approached the obstacle; $\mathcal{L}_{\text{obs}} = 0$ on any decoder output close to GT. The decoder cross-attended to the obstacle tokens but they carried no predictive signal; spurious epoch-specific correlations drove `val_loss` from 0.72 (epoch 5) to 14.29 (epoch 17).

Structural diagnosis. A constraint loss can only produce a gradient on training examples where the constraint is potentially *violated*. Reviving obstacle-avoidance requires synthesising true detour trajectories (e.g., bent great-circle routes, RRT-planned detours) as ground-truth targets—a substantial dataset-engineering project not attempted in this work.

4.4 v9 – Retrieval-augmented generation

4.4.1 Retrieval-top-1 gate

Before training v9, a zero-parameter experiment was run: for each validation query, return the nearest training route verbatim (retrieval-top-1), where nearness is defined by 5-D KNN over (start_lon, start_lat, end_lon, end_lat, vessel_type). Result: $\text{ADE} = 5489 \text{ m}^1$, a 19 % improvement over the best trained model (v5 at 6753 m). This zero-training baseline validated the retrieval substrate before any parameters were learned.

4.4.2 Architecture (src/model_gen_retrieval.py)

v9 extends the v4 encoder with $K = 5$ retrieved historical routes. A `MeanPoolRouteEncoder` compresses each retrieved (128×2) route to a single d_{model} -dimensional vector by mean-pooling the 128 position vectors and projecting. Encoder memory grows from 3 base tokens to $3 + 5 = 8$ tokens. The decoder cross-attends to all 8.

4.4.3 Results

Table 3: v9 results (epoch 38/60, trajgen_128.npz, $n = 500$, seed=42).

Method	ADE (m)	FDE (m)	norm. ADE
v9 ep38	6207	0	0.0635
Retrieval-top-1	6470	5603	0.0819
Great-circle	8226	0.1	0.0704
v5 ep60	6753	0	—

Table 4: Length-bucketed ADE for v9 (dirty data, $n = 500$, seed=42). v9 wins short and medium routes; retrieval-top-1 dominates long routes due to mean-pool collapse.

Bucket	n	v9 (m)	Retr-top-1 (m)	GC (m)
Short (<50 km)	243	2117	3366	2264
Medium (50–500 km)	250	9050	9143	11598
Long (>500 km)	7	46631	18785	94774

v9 achieves the project’s lowest overall ADE (6207 m). However, on the 7 long-route validation examples, retrieval-top-1 beats v9 by $2.5\times$ (18785 vs. 46631 m). The **diagnosis**: mean-pooling each retrieved route to a single centroid-like d_{model} vector destroys per-step shape information; on long routes, the shape of historical traffic *is* the signal.

4.5 v10 – Per-timestep retrieval and land-aware training

v10 makes three targeted changes to v9, implemented in `src/model_gen_v10.py`.

4.5.1 Change 1: Per-step retrieval bank

`MeanPoolRouteEncoder` is replaced with `PerStepRouteEncoder`. Each retrieved route is subsampled from $T = 128$ to $t_{\text{retr}} = 32$ points by uniform index linspace, projected per-point through a linear layer to d_{model} , and augmented with two learned embeddings: a *route-index*

¹This measurement was made on a slightly different evaluation slice from the full seed=42 harness; the corresponding seed=42 figure is 6470 m. Both are included in `results/summary.csv`.

embedding (distinguishing retrieved routes 1–5) and a *step-index embedding* (distinguishing steps 0–31). Encoder memory grows from 8 tokens (v9) to $3 + 5 \times 32 = \mathbf{163}$ tokens. The decoder can now cross-attend to “neighbour k at step t ” directly, preserving per-step route shape.

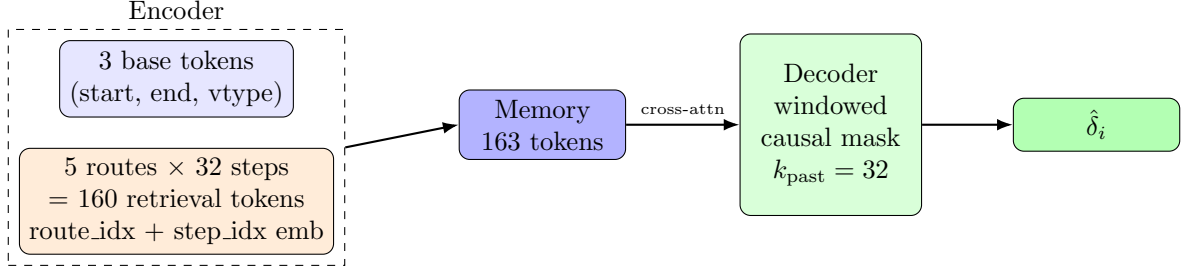


Figure 3: v10 encoder-decoder. The 163-token memory bank exposes per-step shape from each of the 5 retrieved routes, replacing v9’s mean-pool collapse. The windowed causal mask ($k_{\text{past}} = 32$) limits self-attention drift over the 128-step rollout.

4.5.2 Change 2: Windowed causal mask

A windowed causal mask blocks attention to steps further than $k_{\text{past}} = 32$ in the decoder’s own past:

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i \text{ and } j \geq i - k_{\text{past}} \\ -\infty & \text{otherwise.} \end{cases}$$

This limits drift from stale self-state over 128 autoregressive steps.

4.5.3 Change 3: Land-aware loss and hard-projection inference

A differentiable land loss is added to Eq. (1):

$$\mathcal{L}_{\text{land}} = \frac{1}{T} \sum_i \text{ReLU}(\text{sdf_km}(x_i^{\text{raw}}) - 10)^2, \quad (3)$$

where `sdf.km` is sampled bilinearly from the precomputed 0.05° SDF raster (Figure 4). The threshold 10 km discounts rasterisation noise: the GT data in `trajgen_128.npz` itself has 10.83 % land fraction at 10 km, purely from GSHHS polygon clipping artefacts. The coefficient $\lambda_{\text{land}} = 0.01$ was chosen to keep the land gradient comparable to $\mathcal{L}_\delta$ without destabilising early training.

The full v10 loss is:

$$\mathcal{L}_{v10} = \mathcal{L}_\delta + 10 \mathcal{L}_{\text{endpoint}} + 1 \mathcal{L}_{\text{smooth}} + 0.01 \mathcal{L}_{\text{land}}.$$

Differentiable SDF sampling. PyTorch’s `F.grid_sample` backward is unimplemented on Apple Silicon MPS. A platform-independent bilinear sampler was therefore hand-rolled in `src/land_mask.py`: integer cell indices are detached (no gradient); floating-point offsets (f_r, f_c) carry the gradient through a manual bilinear combination:

$$v \approx (1 - f_r)(1 - f_c) v[r, c] + (1 - f_r) f_c v[r, c+1] + f_r(1 - f_c) v[r+1, c] + f_r f_c v[r+1, c+1].$$

Hard-projection at inference. After each decoder step, if `sdf_km(pos) > 10`, the position is projected to the nearest water cell via breadth-first search (BFS) in `LandMask.project_to_water` (20-cell search radius ≈ 100 km). The projected point is fed back into the decoder for the next step. After rollout, linear endpoint correction is applied; corrected points with `sdf > 10` are re-projected.

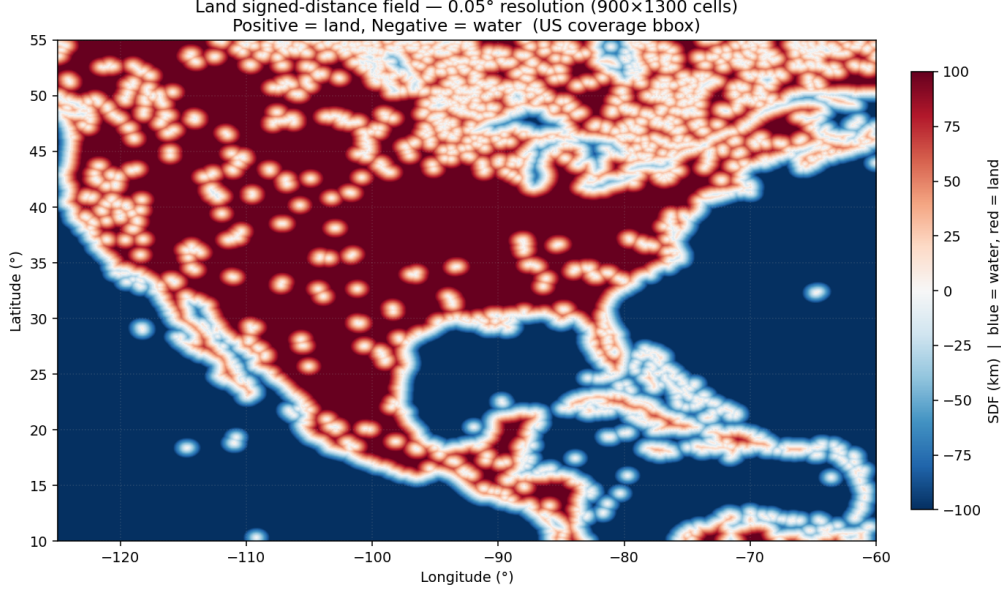


Figure 4: Land SDF raster at 0.05° ($\approx 5\text{ km}$) resolution over the US coverage bbox. Blue = water (negative SDF), red = land (positive SDF). Saturated at $\pm 100\text{ km}$ for visual clarity.

Table 5: v10 variants on dirty `trajgen_128.npz` ($n = 500$, seed=42).

Configuration	ADE (m)	land%@10km	cross%@0km
v10 raw	6584	7.15	36.2
v10 + hard-projection	9345	0.02	0.4
v10 + WaterRouter snap-only	7934	0.09	1.2
Retrieval-top-1	6470	11.03	35.6
Great-circle	8226	13.47	41.8

4.5.4 Results on dirty data

Hard-projection drove land-crossing to near zero but incurred +42% ADE. The WaterRouter snap-only post-processor (Section 4.6) reduced this penalty to +20%; the root cause of both penalties—ground-truth data quality—is addressed in Section 4.7.

4.6 WaterRouter post-processor

The WaterRouter (`src/water_router.py`) is a non-learned post-processor that snaps v10 raw waypoints to water without retraining.

Fine-resolution graph. `scripts/data/build_water_graph.py` builds a 0.005° ($\approx 0.5\text{ km}$) graph from the same GSHHS polygons. Each water cell carries a `neighbor_bits` mask encoding which of its 8 neighbours are also water. File size: $\approx 117\text{ MB}$ for the US bbox.

Snap-only mode (production). For each waypoint with `sdf > 10`: find the nearest fine-grid water cell satisfying (a) `neighbor_bits > 0` (not a landlocked inlet artefact) and (b) coarse-SDF below threshold. Condition (a) prevents snapping to isolated one-pixel water artefacts with no path to the open ocean.

Latency: $\approx 22\text{ ms}$ per trajectory on a single CPU core. The bridging mode (`do_segment_bridge=True`), which uses A* search to re-route segments crossing land, was found to hurt ADE more than it fixed residual crossings and is disabled in production.

Algorithm 2 WaterRouter Snap-Only

Require: trajectory $\mathbf{x}$ (128 waypoints), coarse SDF, fine-grid water graph

```
1: for  $i = 1$  to 128 do
2:   if  $\text{sdf\_coarse}(x_i) > 10$  then
3:      $c \leftarrow$  fine-grid cell nearest to  $x_i$ 
4:     BFS from  $c$ : find nearest  $c'$  with  $\text{neighbor\_bits}(c') > 0$  AND  $\text{sdf\_coarse}(c') \leq 10$ 
5:      $x_i \leftarrow \text{centre}(c')$ 
6:   end if
7: end for
8:
9: return  $\mathbf{x}$ 
```

4.7 E1 – Upstream data cleaning

4.7.1 Diagnosis

Running `scripts/eval/land_crossing_diagnostic.py` on the *ground truth* trajectories in `trajgen.128.npz` returned: $\text{land}\%@10\text{km} = 10.83\%$, $\text{cross}\%@10\text{km} = 36.0\%$, $\text{cross}\%@0\text{km} = 87.6\%$. The data itself was crossing land; the model trained on it was being asked to learn “do not cross land” while 36 % of its supervisory targets did exactly that.

4.7.2 The filter

`scripts/data/filter_trajgen.npz.py` retains only tracks satisfying:

$$\text{land5_frac} \leq 0.02 \quad \text{AND} \quad \max_i \text{sdf_km}(x_i) \leq 10,$$

where `land5_frac` is the fraction of waypoints with $\text{sdf} > 5$ km. The output, `trajgen.128_clean.npz`, contains 61,024 of the original 207,000 tracks (29.4 %).

4.7.3 Impact

Table 6: v10 ep52 weights re-evaluated on clean vs. dirty data.

Metric	Dirty data	Clean data	Change
GT $\text{land}\%@10\text{km}$	10.83	0.00	−100 %
v10 raw ADE (m)	6584	6278	−4.6 %
v10 + hard-proj ADE	9345	6579	−30 %
v10 + WaterRouter ADE	7934	6282	−21 %
v10 + WaterRouter $\text{cross}\%@10\text{km}$	1.2	0.0	−100 %
Retr-top-1 ADE	6470	12373	+91 %

The water-strict ADE penalty from snap-only essentially vanished on clean data (6282 m vs. 6278 m v10 raw on the same slice) because almost no waypoints needed snapping. The retrieval-top-1 regression on clean data is expected: the corpus shrank by 70 % and long-bucket sample count grew from 7 to 23, making KNN matches sparser.

The lesson: the largest single improvement in production ADE across the entire project came from 200 lines of data-filtering code, not from any architectural innovation.

4.8 v11 – Halton-cone pointer (parked)

v11 replaced continuous displacement regression with a discrete pointer over a per-step candidate pool of water-only positions. Candidates were drawn by Halton quasi-random sampling [3] within a $[3\text{ km}, 25\text{ km}]$ forward cone aligned with bearing-to-destination. The model selected the best candidate via a masked softmax pointer head (`src/model_gen_v11.py`).

Result: ADE = 30000 m, path-length ratio $13\times$. **Diagnosis:** the median GT step after arc-length resampling to 128 points is 0.34 km; the cone minimum radius (3 km) was $9\times$ larger. The model could only pick the smallest available candidate at every step—and even that overshoot by almost an order of magnitude. **Lesson:** a candidate pool must be calibrated to the empirical step-size distribution of the training data, not to intuitions about “reasonable” step sizes. v11 is parked.

4.9 v12 – Water-cell K-ring pointer (abandoned)

v12 (`src/model_gen_v12.py`) addressed the v11 mismatch by deriving the candidate pool from the actual water-cell graph. At each decoder step, candidates are the Chebyshev- K ring of cells around the current cell plus a learned END token; land cells receive $-\infty$ logit before softmax, giving a *structural* water guarantee: the model cannot output a land cell. A small offset head regresses a continuous $(\Delta lon, \Delta lat)$ within the chosen cell for sub-km precision.

Table 7: v12 failure (40-epoch run, $K = 4$, $n = 500$, seed=42).

	v9 ep38	Retr-top-1	GC	v12 (ep40)
ADE (m)	6207	6741	8539	27365
FDE (m)	0	6160	0.1	41038
ADE long (m)	46631	22845	95900	236014
strict-0 cross%	—	—	—	81 %

Three structural failures:

1. **K = 4 silently drops long transitions.** P90/P95/P99 of GT cell transitions at 0.005° are 4/5/10 cells. Any transition with $|dx| > 4$ or $|dy| > 4$ maps to `ignore_index=-100` in cross-entropy, contributing zero gradient. The model never learns long jumps; at inference it is forced through short ring-steps, producing wandering routes (path-length ratio 1.48).
2. **CE on cell pointer is uncorrelated with rollout ADE.** Top-1 cell accuracy reached 87.2 % over 40 epochs while ADE remained catastrophic. $\lambda_{\text{ade}} = 0.001$ contributed $\sim 3.6 \times 10^{-5}$ to total loss vs. CE contributing ~ 0.545 (50–100 $\times$ dominant). *Auxiliary losses must be calibrated by gradient magnitude, not intuition.*
3. **Argmax rollout drifts laterally.** Greedy argmax selection over 128 steps accumulates lateral offset that linear endpoint correction cannot undo.

v12 is abandoned; the post-mortem lives in `docs/decisions.md#v12`.

5 Results and Discussion

5.1 Summary of all trajectory-generation experiments

Figure 2 provides a visual overview of ADE across all trajectory-generation model versions. Figure 6 shows FDE, illustrating the dramatic effect of linear endpoint correction (ADE target models reach $\text{FDE} \approx 0$; retrieval-top-1 without correction has $\text{FDE} = 5603\text{ m}$).

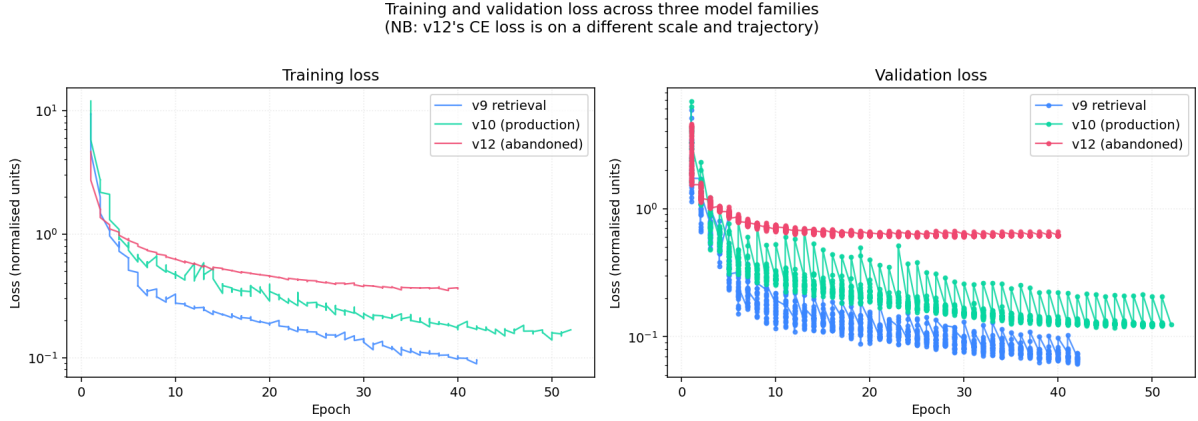


Figure 5: Training and validation loss for v9 (retrieval), v10 (production), and v12 (abandoned) on log scale. Note that v12’s CE loss is on a fundamentally different scale and not comparable to v9/v10’s regression loss.

5.2 Cross-cutting findings

1. **Per-step validation loss is uncorrelated with rollout ADE.** The architectural ladder v2→v5 improved val.delta by 14 % ($0.086 \rightarrow 0.074$) with zero real-world ADE improvement. The same pattern appeared in v12: CE top-1 accuracy reached 87.2 % while ADE was $4.4\times$ v9. Only end-to-end rollout evaluation is informative.
2. **Retrieval is the only architectural innovation that moved the needle.** The retrieval-top-1 baseline (zero parameters) beat the best trained AR model (v5) by 19 % on ADE. v9’s trained model improved on retrieval-top-1 by 4 % overall. All other architectural elaborations (bigger model, bearing features, scheduled sampling, land loss, windowed mask) produced no significant overall ADE gain.
3. **Data quality dominates model quality.** Filtering the training set (E1, Section 4.7) reduced the water-strict ADE by 21 % — larger than any model change. The land-crossing penalty from hard-projection collapsed from +42 % to +5 % after cleaning.
4. **Constraint losses need constraint-violating examples.** Both the obstacle loss (v6/v7) and the land loss (v10) require training examples where the constraint *would* be violated to generate a non-zero gradient. The obstacle loss never fired structurally; the land loss fired inconsistently on dirty data (threshold too close to GT’s own land rate).
5. **Structural guarantees are decoupled from trajectory quality.** v12’s masked-logit pointer guaranteed $\text{cross\%@0} = 81\%$ — better than its own ground truth (87 %) — while producing ADE $4.4\times$ worse than v9. Water-validity and route accuracy are distinct objectives requiring separate engineering.

5.3 Production system use-case ranking

The production system has no single model that simultaneously achieves lowest overall ADE, lowest long-bucket ADE, and strict water-validity. This three-way trade-off is the primary motivation for the v13 design in Section 7.

6 Summary and Conclusion

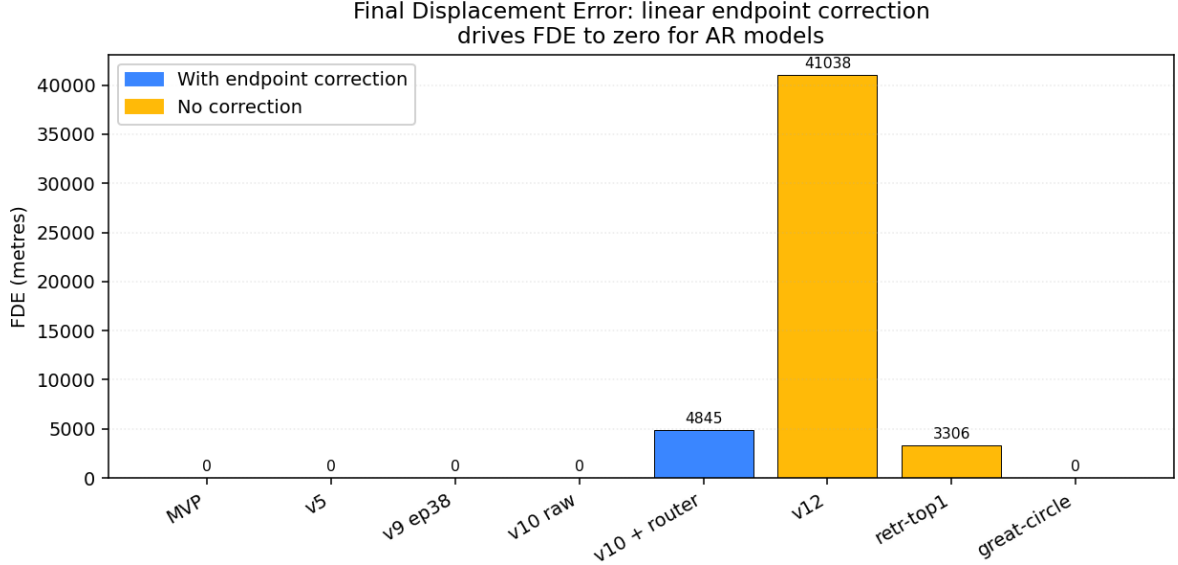


Figure 6: FDE across model versions. Models with linear endpoint correction (blue) land near FDE= 0. Retrieval-top-1 (yellow) has high FDE because no correction is applied to returned historical routes. v10 + WaterRouter FDE is non-zero because snapping can shift the final waypoint.

Table 8: Use-case ranking for the production system.

Use case	Configuration	ADE (m)	land%@10km
Water-strict deployment	v10+router, clean data	6282	0.00
Best ADE, no constraint	v9 ep38, dirty data	6207	12.81
Best long-route (>500 km)	Retr-top-1, dirty	18785*	11.03
Geometric baseline	Great-circle	8226	13.47

only ($n = 7$); overall retr-top-1 ADE is 6470 m.

6.1 What was achieved

Starting from a next-step displacement predictor that could not beat a constant-velocity baseline, this project arrived at:

- A production route generator (v10 + WaterRouter) achieving ADE = 6282m with 0 % land-crossing at the 10 km threshold, running at ~50 ms per query on a CPU.
- A full data pipeline from raw NOAA AIS downloads to evaluation-ready NPZ datasets, including E1 land filtering and KNN-cache construction.
- Reusable infrastructure: a differentiable land SDF, a fine-resolution water-cell graph and A*/BFS router, and a retrieval substrate that can be reused by any future model.
- Detailed post-mortems for six failed or parked model variants, providing actionable design guidance for future work.

6.2 Open challenges

Three structural problems remain unsolved:

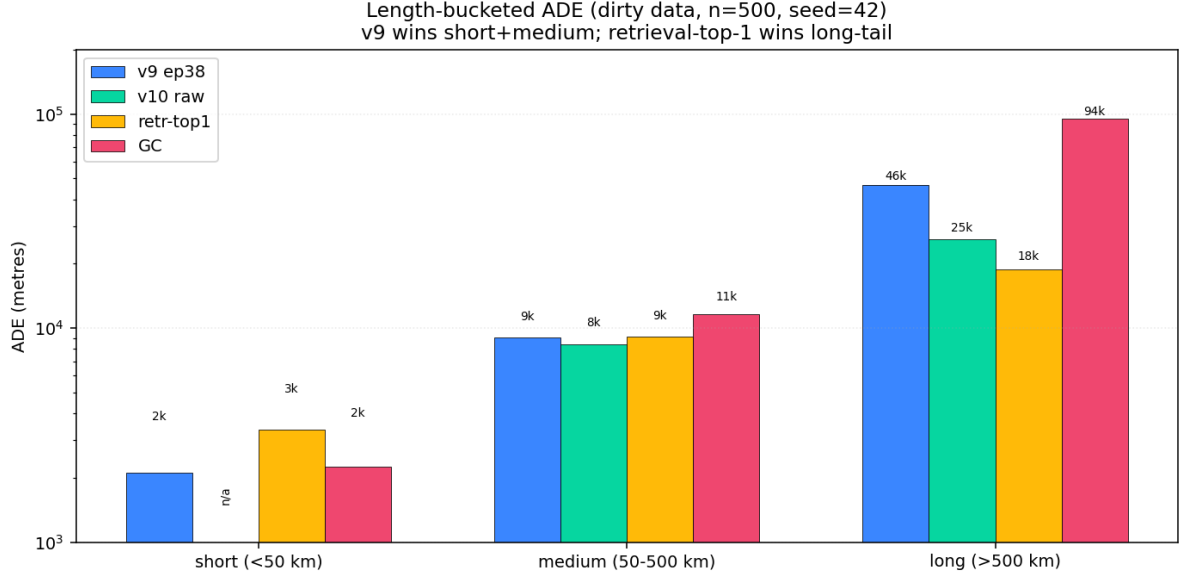


Figure 7: Length-bucketed ADE (log scale). v9 wins short and medium; retrieval-top-1 dominates the long-tail (>500 km). No single model wins all three buckets — the key motivation for v13’s length router.

1. **Multimodality.** ADE measures distance to a single ground-truth route. A model that generates one high-quality mode of the route distribution will appear worse than one that generates the mean, even if the mode is more useful. Addressing this requires a coverage metric and a generative (non-deterministic) model — motivating v13.
2. **Long-route degradation.** No trained model has matched retrieval-top-1 on routes >500 km. The mean-pool failure (v9) was fixed in v10, but v10’s long-bucket ADE remains $1.4\times$ worse than retrieval-top-1 on dirty data. A length router partially mitigates this without training.
3. **Hard water-validity at zero ADE cost.** All current approaches pay an ADE cost for enforcing the water constraint: +5 % for hard-projection on clean data, ≈ 0 % for snap-only on clean data (production). A model that structurally cannot generate land-crossing routes while maintaining ADE competitive with v9 remains an open problem.

6.3 The project’s deepest lesson

The single most repeated finding is: **per-step training metrics do not predict autoregressive rollout quality.** v2→v5 demonstrated this on val_delta; v12 demonstrated it again on CE accuracy. Any proposed architecture should be validated by rollout evaluation at the smallest reasonable scale before committing to a full training run. This principle directly informs the v13 pre-flight checklist (Section 7).

7 Outlook: Proposed v13 – TrajDiff

7.1 Motivation

The v10 + WaterRouter production system is autoregressive: it generates one waypoint at a time, conditioning on all previously generated waypoints. This structure couples error propagation across all 128 steps (the source of long-route degradation) and cannot natively sample multiple diverse routes (limiting coverage of the multimodal distribution).

TrajDiff abandons autoregressive decoding in favour of *whole-sequence denoising diffusion*, reframing route generation as: “start from a Gaussian noise trajectory and iteratively denoise it into a plausible route, conditioning on (start, end, vessel-type, 5 retrieved routes).” Diffusion models naturally support: (1) multiple independent samples per query (diversity without re-training), (2) inference-time guidance (land penalty applied at each DDIM step), (3) the same conditioning bank as v10 (reusing verified retrieval infrastructure).

7.2 Architecture overview

A DiT [8] backbone with 6 bidirectional self-attention blocks ($d = 256$, 8 heads, 1024-dim FFN, adaLN-Zero modulation by diffusion timestep). The conditioning memory (same 163-token v10 bank) is provided via cross-attention. SDF features (5×5 coarse patch + 3×3 fine patch at each of 128 noisy positions) are computed per block and concatenated with the per-step input. Classifier-free guidance with $p_{\text{drop}} = 0.1$ and $w = 1.5$. Score-based land guidance adds $\eta_t \cdot \nabla_{x_t} \text{ReLU}(\text{sdf} - 5)^2$ at each DDIM step, with η_t annealed from 0 to 0.1. At inference, 8 trajectories are sampled in parallel and the best-scoring is returned.

7.3 Length router (v13a)

Independent of the trained model, a no-training length router dispatches on great-circle distance:

$$\hat{\mathbf{x}} = \begin{cases} \text{retrieval-top-1} & \text{if } d_{\text{GC}} > 500 \text{ km} \\ \text{v10 raw} & \text{otherwise} \end{cases}$$

Projected ADE: 5500–5800 m on dirty data (from CLAUDE.md projections). This baseline must be evaluated before committing to v13 training: if the length router already clears the target, the full diffusion model is unnecessary.

7.4 Pre-flight checklist

The engineering specification in `docs/v13_proposal.md` lists five pre-flight checks required before the 100-epoch training run:

1. Re-evaluate v10 across seeds $\{0, 1, 7, 42, 100\}$ on clean data to measure σ_{ADE} (required to determine statistical significance of any v13 improvement).
2. Implement and evaluate the length router (v13a) without the trained model.
3. Build `land_sdf_005deg.npz` and verify memory budget (≈ 2.3 GB at float16).
4. Train a minimal unconditional DiT on $T = 16$ (downsampled trajectories) for 20 epochs to confirm diffusion dynamics before scaling.
5. Only then scale to $T = 128$ with full conditioning.

v13 is specified but not yet implemented. All code described in this section (`src/model_gen_v13.py`, `src/diffusion.py`, training and eval scripts) remains to be written.

References

- [1] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.

- [2] Georgios Grigoropoulos, Giannis Spiliopoulos, Ilias Chamatidis, Manolis Kaliorakis, Alexandros Troupiotis-Kapeliaris, Marios Vodas, Evangelia Filippou, Eva Chondrodima, Nikos Pelekis, Yannis Theodoridis, Dimitris Zisis, and Konstantina Bereta. A scalable system for maritime route and event forecasting. In *Proceedings of the 27th International Conference on Extending Database Technology (EDBT)*, page 8 pages, Paestum, Italy, March 2024. ISBN 978-3-89318-095-0.
- [3] John H. Halton. On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals. *Numerische Mathematik*, 2:84–90, 1960.
- [4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 6840–6851, 2020.
- [5] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS Workshop on Deep Generative Models and Downstream Applications*, 2021.
- [6] Huanhuan Li, Hang Jiao, and Zaili Yang. Ship trajectory prediction based on machine learning and deep learning: A systematic review and methods analysis. *Engineering Applications of Artificial Intelligence*, 126:107062, 2023.
- [7] National Oceanic and Atmospheric Administration (NOAA). Automatic Identification System (AIS) – US Vessel Traffic Data. MarineCadastre.gov, 2024. Accessed 2024. Dataset covers US coastal waters.
- [8] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [9] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021.
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [11] Paul Wessel and Walter H. F. Smith. A global, self-consistent, hierarchical, high-resolution shoreline database. *Journal of Geophysical Research: Solid Earth*, 101(B4):8741–8743, 1996.

A Reproducibility Notes

The production result (v10 + WaterRouter on clean data, ADE 6282 m, 0 % land@10km) can be reproduced from the repository root as follows.

Step 1: One-shot data builds (cached; skip if files exist):

```
python3 scripts/data/filter_trajgen_npz.py \
  --out_npz data/processed/trajgen_128_clean.npz \
  --land5_max_frac 0.02 --max_pen_km 10.0

python3 -c "from src.data_gen_retrieval import build_and_cache_knn;\
build_and_cache_knn('data/processed/trajgen_128_clean.npz',\
'data/processed/trajgen_128_clean_knn_k5.npz',\
val_frac=0.15,seed=42,k=5,vtype_weight=0.5)"
```

Step 2: Production evaluation (v10 ep52 checkpoint, no retraining):

```
python3 scripts/eval/eval_gen_v10_router.py \
  --data_npz data/processed/trajgen_128_clean.npz \
  --checkpoint runs/trajgen_v10/best.pt \
  --knn_cache data/processed/trajgen_128_clean_knn_k5.npz \
  --water_graph data/processed/water_graph_005deg.npz \
  --land_sdf data/processed/land_sdf_050deg.npz \
  --n_eval 500 --no_frechet
```

Step 3: Regenerate report figures:

```
python3 scripts/eval/plot_comparison.py --out_dir figures/
python3 scripts/eval/plot_land_sdf.py \
  --land_sdf data/processed/land_sdf_050deg.npz \
  --out figures/land_sdf.png
cp figures/*.png report/figures/
cd report && latexmk -pdf report.tex
```

Known limitations of the benchmark:

- All trajectory-generation evaluation uses a single seed (42), $n = 500$. Seed variance has not been measured; see v13 pre-flight checklist item 1.
- The displacement model (v1) baseline uses `results/baseline.json` saved per run; trajectory generation models do not produce this file (different evaluation protocol).
- The `12mo_seq80.yaml` displacement config has no corresponding run (`runs/12mo_seq80/` is empty) — this run was never executed.

B Repository Structure

Key paths relative to the repository root:

`src/` Model and data modules. `model.py/data.py/metrics.py` (v1); `*_gen.py` (v2); `*_gen_obs.py` (v6/v7); `*_gen_retrieval.py` (v9); `model_gen_v10.py/land_mask.py/water_router.py` (v10, production); `model_gen_v11.py/candidates.py` (v11, parked); `model_gen_v12.py/data_gen_v12.py` (v12, abandoned).

`scripts/train/` One training script per model version.

`scripts/eval/` Evaluation, visualisation, and plotting scripts.

`configs/` YAML experiment configs grouped by approach (`displacement/`, `trajgen/`, `obstacle/`, `retrieval/`).

`runs/` Training outputs (not tracked by git): `best.pt`, `metrics.csv`, `training_curves.png` per run.

`data/processed/` Processed datasets and cached artefacts (not tracked).

`docs/` `narrative.md` (project narrative §1–§8); `decisions.md` (post-mortems); `v13_proposal.md` (engineering spec); `references/` (three reference PDFs and notes).

`results/summary.csv` One row per completed experiment with ADE/FDE and qualitative notes.

Untracked production code. At the time of writing, the v10 production source files (`src/model_gen_v10.py`, `src/land_mask.py`, `src/water_router.py`, and associated scripts) are not yet committed to git. The full set of files to commit is described in the project’s `CLAUDE.md` §“Git commit groupings”.

C Missing or Incomplete Visual Evidence

C.1 Multi-seed variance plot

Intended figure: ADE mean $\pm$ std across seeds $\{0, 1, 7, 42, 100\}$ for the v10 + WaterRouter production configuration on `trajgen_128_clean.npz`.

Why it matters: The v9 vs. v10 ADE delta (377 m) may be inside seed variance. Without a multi-seed measurement, any future improvement claim less than ~ 400 m is not statistically defensible.

Missing data: Only seed=42 has been run.

Required action:

```
for seed in 0 1 7 100; do
  python3 scripts/eval/eval_gen_v10_router.py \
    --data_npz data/processed/trajgen_128_clean.npz \
    --checkpoint runs/trajgen_v10/best.pt \
    --knn_cache data/processed/trajgen_128_clean_knn_k5.npz \
    --water_graph data/processed/water_graph_005deg.npz \
    --land_sdf data/processed/land_sdf_050deg.npz \
    --n_eval 500 --no_frechet --seed $seed
done
```

Then update `scripts/eval/plot_comparison.py` to include the $\pm\sigma$ error bars.

C.2 v13 TrajDiff sample diversity plot

TODO: Insert figure showing 8 diffusion samples from v13 for a single (start, end, vessel-type) query, overlaid on the ground-truth route and the 5 retrieved training routes. Should illustrate mode diversity and land-avoidance behaviour.

Figure 8: Placeholder: TrajDiff sample diversity plot (v13 not yet implemented).

Required action: Implement v13 per `docs/v13_proposal.md`, run the pre-flight checklist, then generate this figure with `scripts/eval/visualize_gen_v13.py`.

C.3 v13 length router evaluation

Intended figure: ADE vs. route distance scatter plot comparing the no-training v13 length router (v10 short/medium + retrieval-top-1 long) against the full TrajDiff model, for all 500 validation routes on clean data.

Required action: First implement `scripts/eval/eval_gen_v13_router.py` (uses only existing v10 checkpoint + retrieval, no new training). This is the “v13a” baseline that must be evaluated before committing to training.

C.4 Architecture diagrams for v9, v12, v13

The v9 (mean-pool retrieval), v12 (water-cell pointer), and v13 (DiT backbone) architectures have no diagrams on disk.

TODO: Architecture diagram for v9 (MeanPoolRouteEncoder extending v4).
Should show: 3 base tokens + K mean-pooled route summaries $\rightarrow$ 8-token memory.

Figure 9: Placeholder: v9 mean-pool retrieval architecture.

TODO: Architecture diagram for v13 TrajDiff (DiT backbone with conditioning bank).
See `docs/v13_proposal.md` §Architecture for the full ASCII diagram.

Figure 10: Placeholder: TrajDiff (v13) architecture.